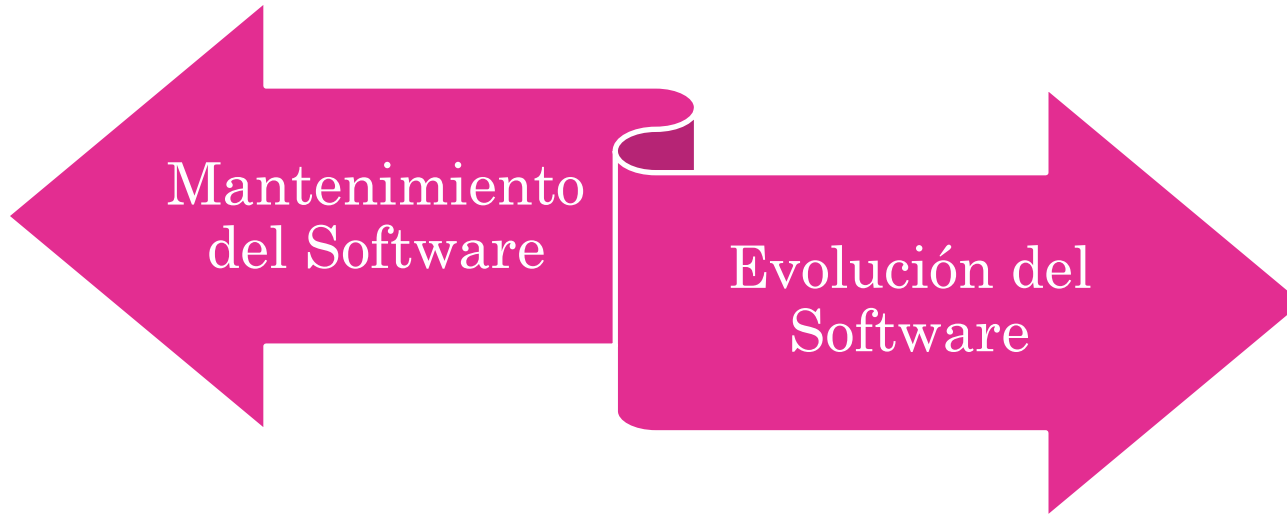


Evolución del Software



El enfoque y los conceptos en el tiempo...



❖ El mantenimiento de software se caracteriza con un iceberg...

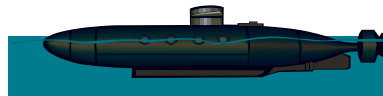
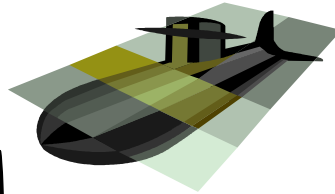
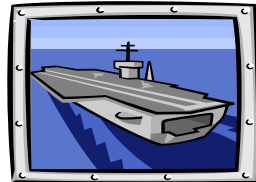
... esperábamos que sólo fuera lo que se veía, pero sabíamos que una enorme masa de problemas y costos yacía bajo la superficie.



- ❖ A principio de los años 70 el iceberg era suficientemente grande como para hundir un portaaviones.



- ❖ En la actualidad podría hundir a toda la armada.



¿Porqué se necesita tanto mantenimiento?



- Software del que dependemos tiene 20 años o más.
- Aún cuando hayan sido contruidos empleando las mejoras técnicas de análisis y diseño (y para la mayoría no fue así) se crearon cuando el tamaño de los programas y el espacio de almacenamiento eran las principales preocupaciones.
- Traslados a nuevas plataformas, cambios de hardware, de sistemas operativos
- Incorporación de mejoras para satisfacer necesidades de los usuarios.
- Y todo eso sin tener en cuenta la **arquitectura!!!**



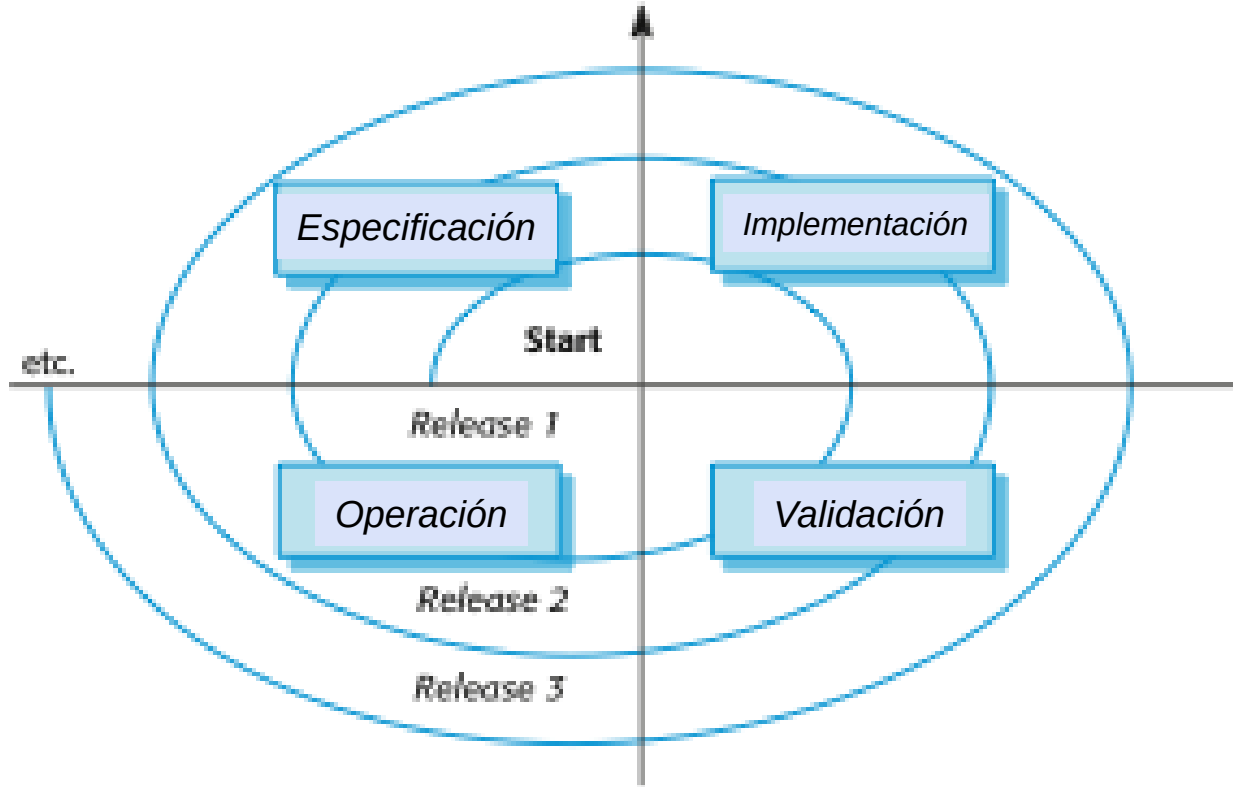
Razones de todo esto...



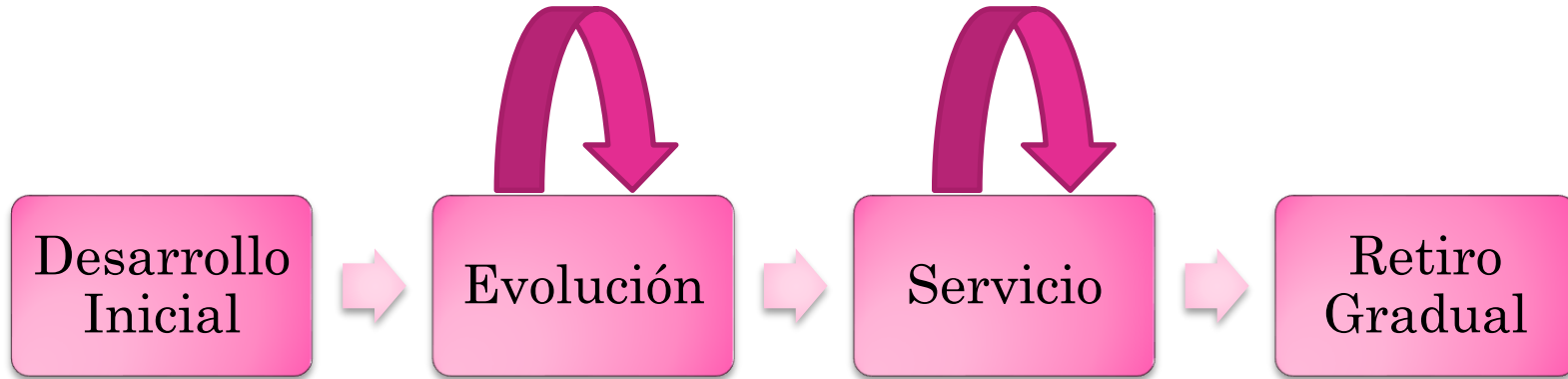
- Arquitecturas muy mal diseñadas
- Mala codificación
- Lógica inadecuada
- Escasa o inexistente documentación



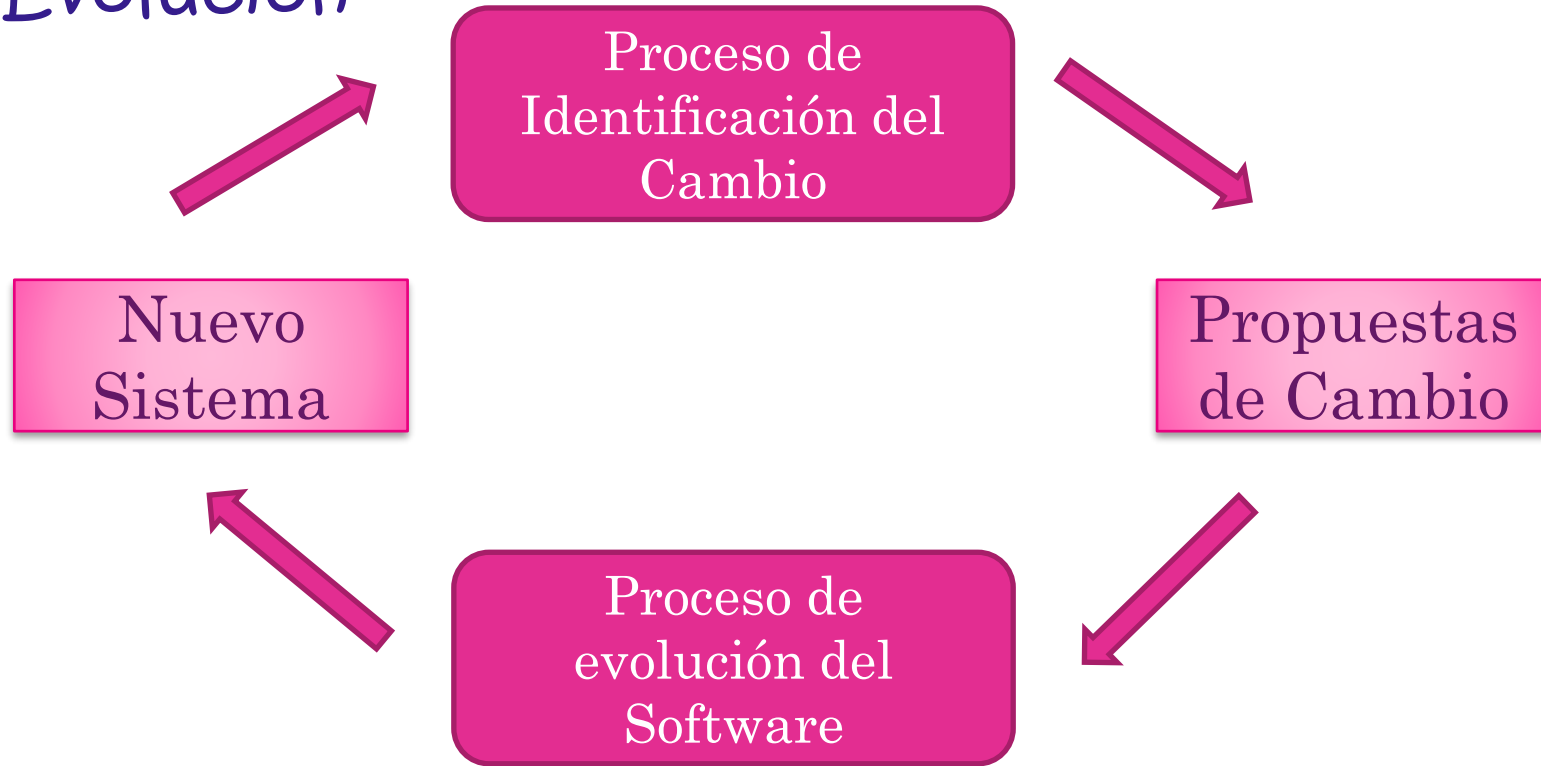
Desarrollo y Evolución del Software



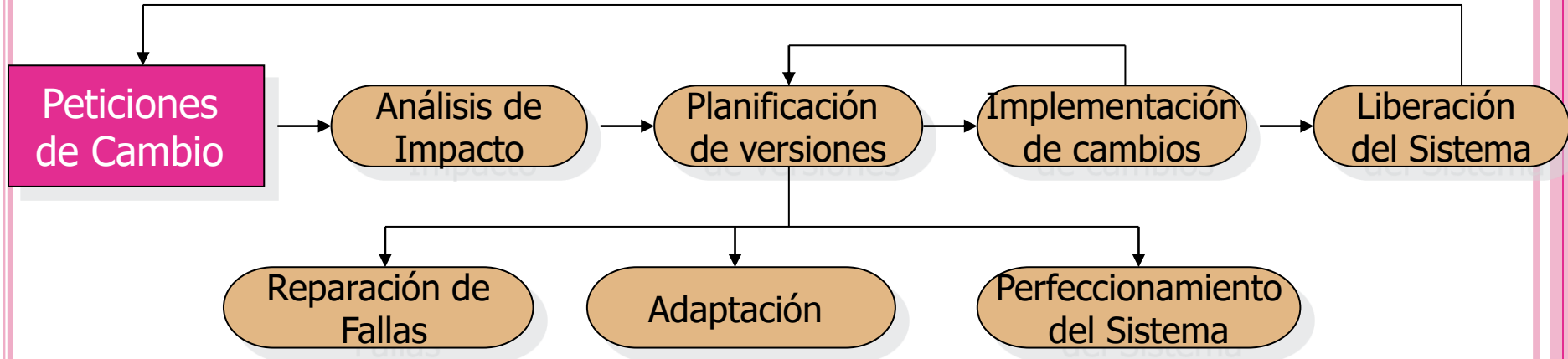
Ciclo de vida en la evolución del software



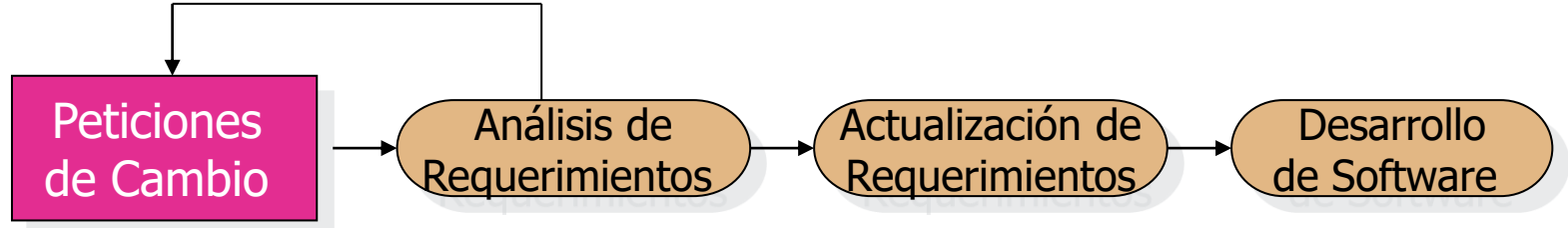
Identificación del Cambio y Procesos de Evolución



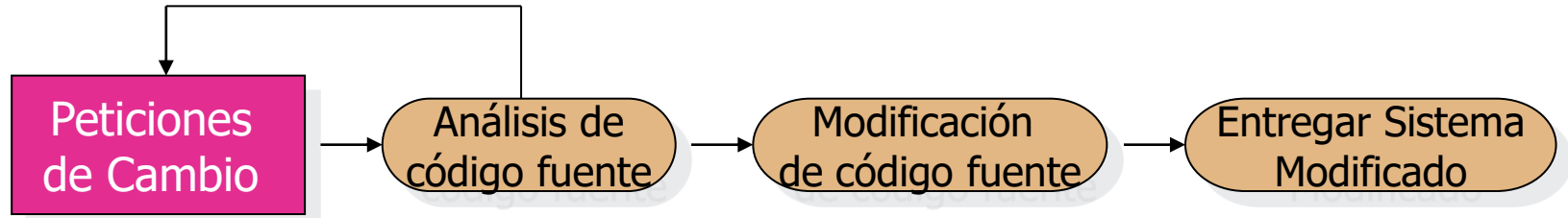
El proceso de evolución del software



Procesos de Cambio



Proceso de Implementación de cambios



Proceso de reparación de fallas de emergencia



Dinámicas de Evolución de los Programas

- ✦ Las dinámicas de evolución del software son el objeto de estudio de los procesos de sistema de cambio.
- ✦ Luego de los principales estudios, **Lehman y Belady** propusieron la existencia de un número de “leyes” que son aplicables a todos los sistemas conforme evolucionen.
- ✦ Existen observaciones sensibles más que leyes. Ellas son aplicables a grandes sistemas desarrollados por grandes organizaciones. Tal vez menos aplicables en otros casos.



La leyes de Lehman

Ley	Descripción
Cambio Continuo	Un programa que es utilizado en un entorno de mundo real debe necesariamente cambiar o se volverá menos útil cada vez, a ese entorno.
Incremento de Complejidad	Como un programa en evolución cambia, su estructura en evolución tiende a ser cada vez más compleja, Deben asignarse recursos extra para preservar y simplificar esta estructura.
Evolución de Grandes Programas	La evolución del programa es un proceso auto regulado. Atributos del sistema tales como tamaño, tiempo entre releases, y el número de reporte de errores, son invariantes, en general para cada release.
Estabilidad Organizacional	A través de la vida de un programa, su ratio de evolución es constante e independiente de los recursos dedicados al desarrollo del sistema.



La leyes de Lehman

Ley	Descripción
Conservación de Familiaridad	A través de la vida el sistema, el cambio incremental en cada release es aproximadamente constante.
Crecimiento Continuo	La funcionalidad ofrecida por los sistemas tiene que crecer continuamente para la mantener la satisfacción del usuario.
Disminución de Calidad	La calidad de los sistemas disminuirá a menos que se modifiquen para reflejar los cambios en su entorno operativo.
Retroalimentación de Sistemas	La evolución de los procesos incorpora realimentación de sistemas multi-agente, multi-ciclo y hay que tratarlos como sistemas de retroalimentación para lograr una mejora significativa del producto

Aplicabilidad de la Ley de Lehman

- ✦ No ha sido establecida completamente.
- ✦ Generalmente es aplicable a sistemas grandes a medida, desarrollados para organizaciones grandes.
- ✦ No es claro como debería modificarse por:
 - Productos de software enlatados
 - Sistemas que incorporan un número significativo de componentes COTS
 - Pequeñas organizaciones
 - Sistemas de tamaño medio

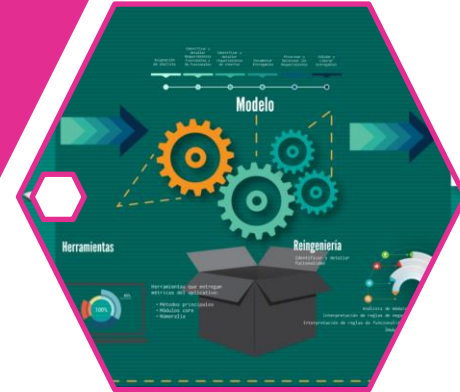


Estrategias de Evolución del Software

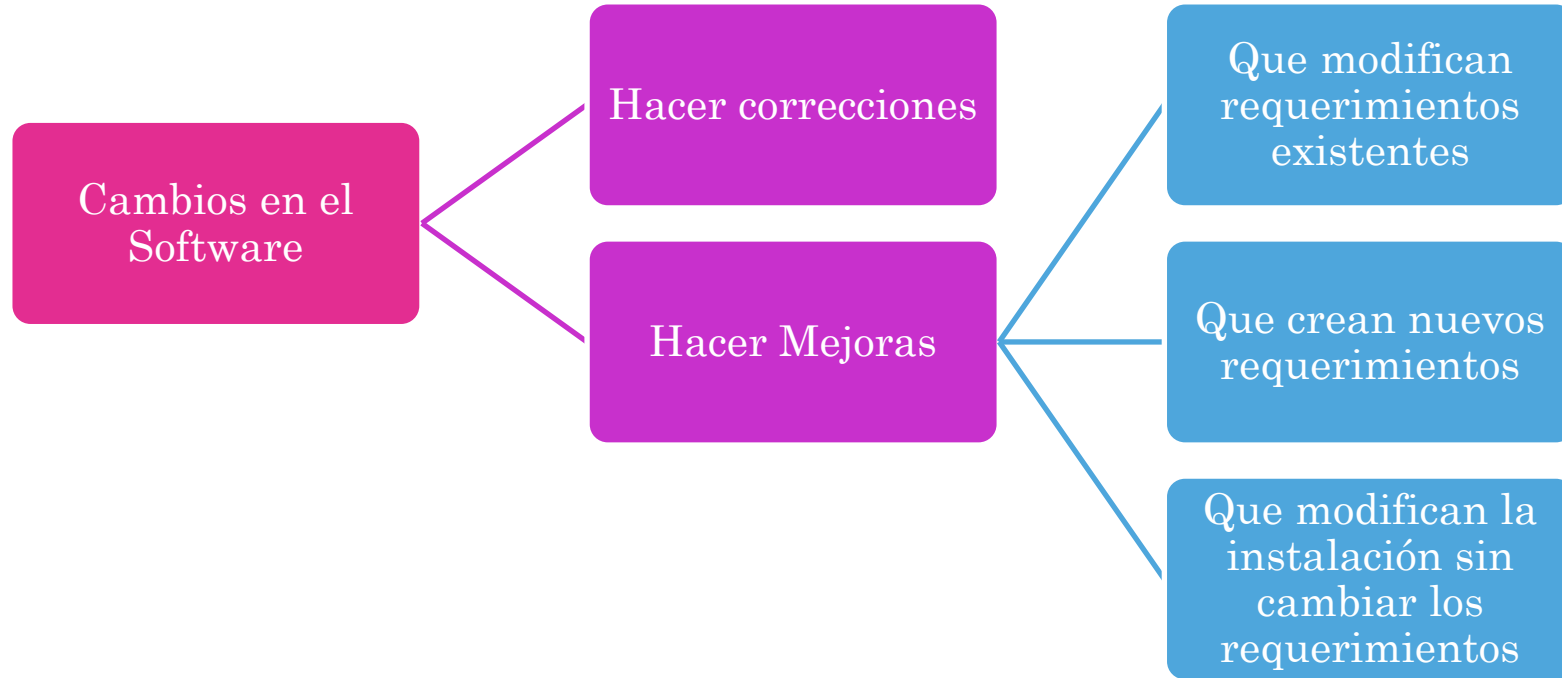


Mantenimiento

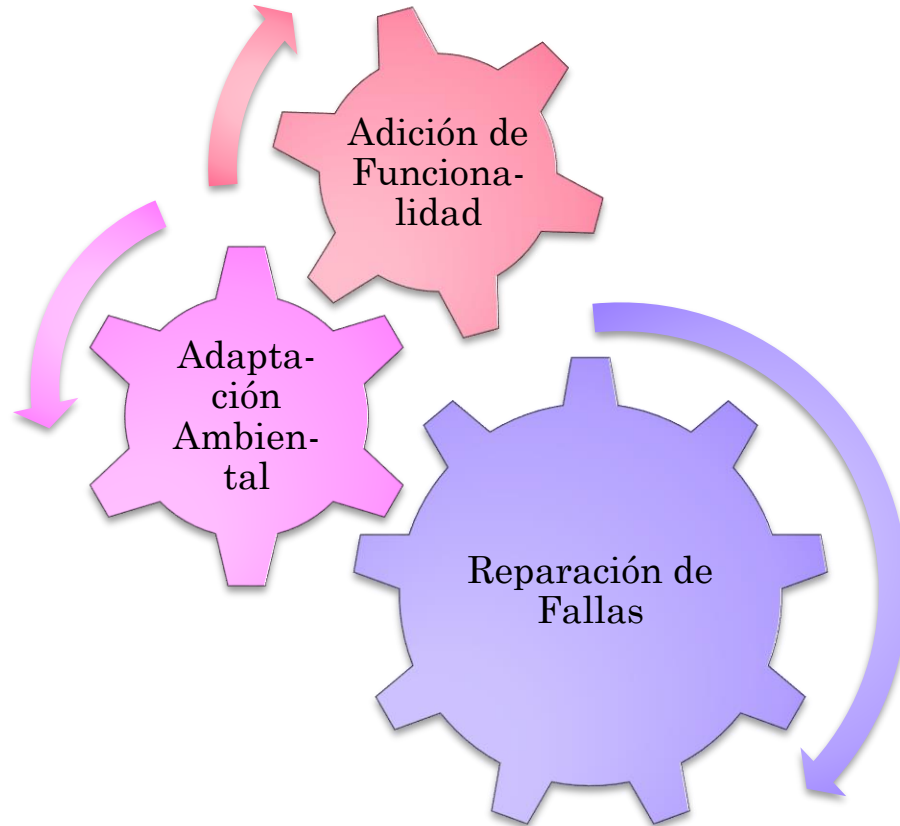
Reingeniería



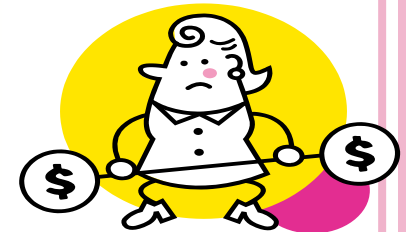
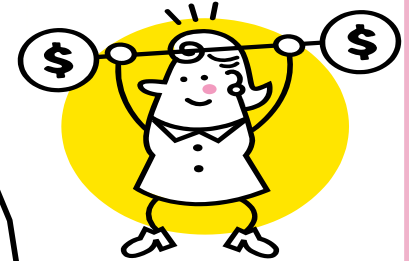
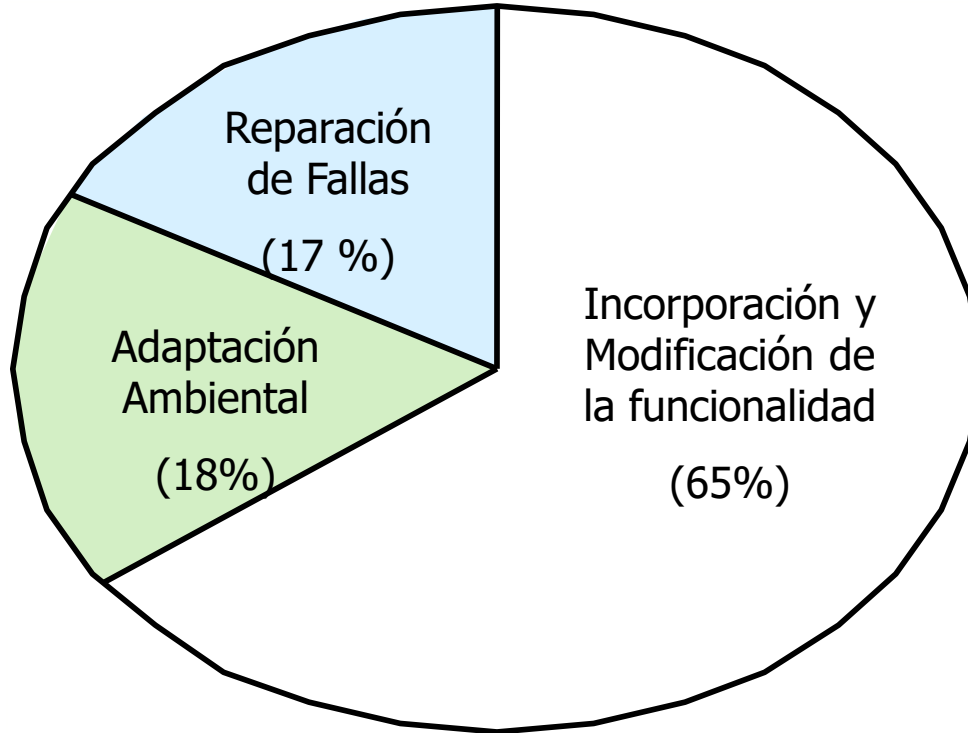
Cambios en el Software



Tipos de Mantenimiento



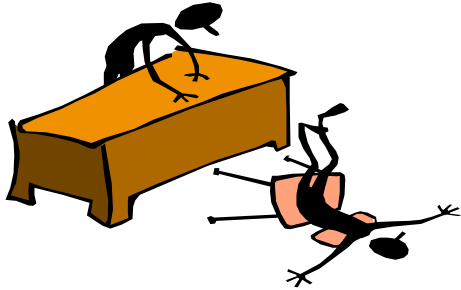
Distribución del esfuerzo de mantenimiento



Factores de Costo de Mantenimiento



Motivación del Personal



- ✦ Relacionar el desarrollo de software a los objetivos organizacionales- mantenimiento racional.
- ✦ Relacionar recompensas a la performance organizacional.
- ✦ Integrar mantenimiento con desarrollo.
- ✦ Crear un presupuesto discrecional para mantenimiento preventivo.
- ✦ Planificar el mantenimiento en las etapas tempranas del proceso de desarrollo.
- ✦ Planificar el gasto de esfuerzo en programas de mantenibilidad.



Predicción de Cambios



La predicción del número de pedidos de cambio requiere comprender las relaciones entre el sistema y su entorno.



Los sistemas fuertemente acoplados requieren cambios cada vez que el ambiente cambia.



Los factores que influyen en esta relación son:

Número y complejidad de las interfaces del sistema.

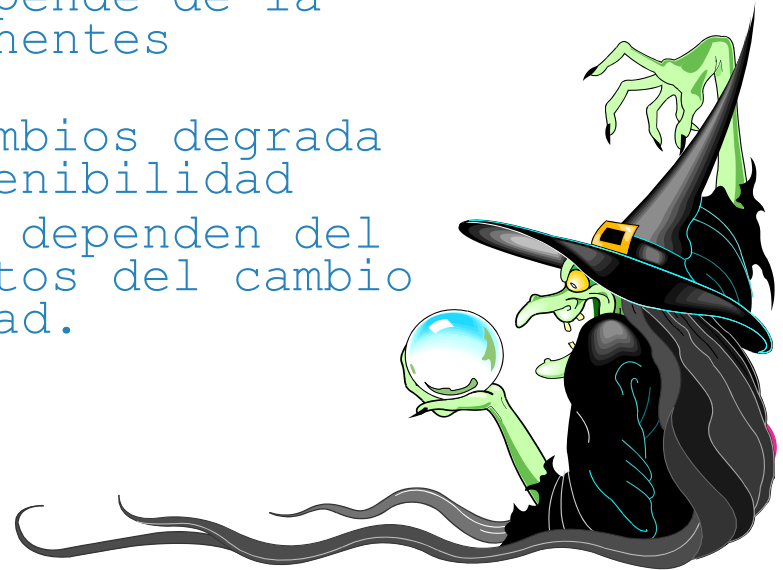
Número de requerimientos volátiles del sistema.

Los procesos de negocio que utilizan el sistema.

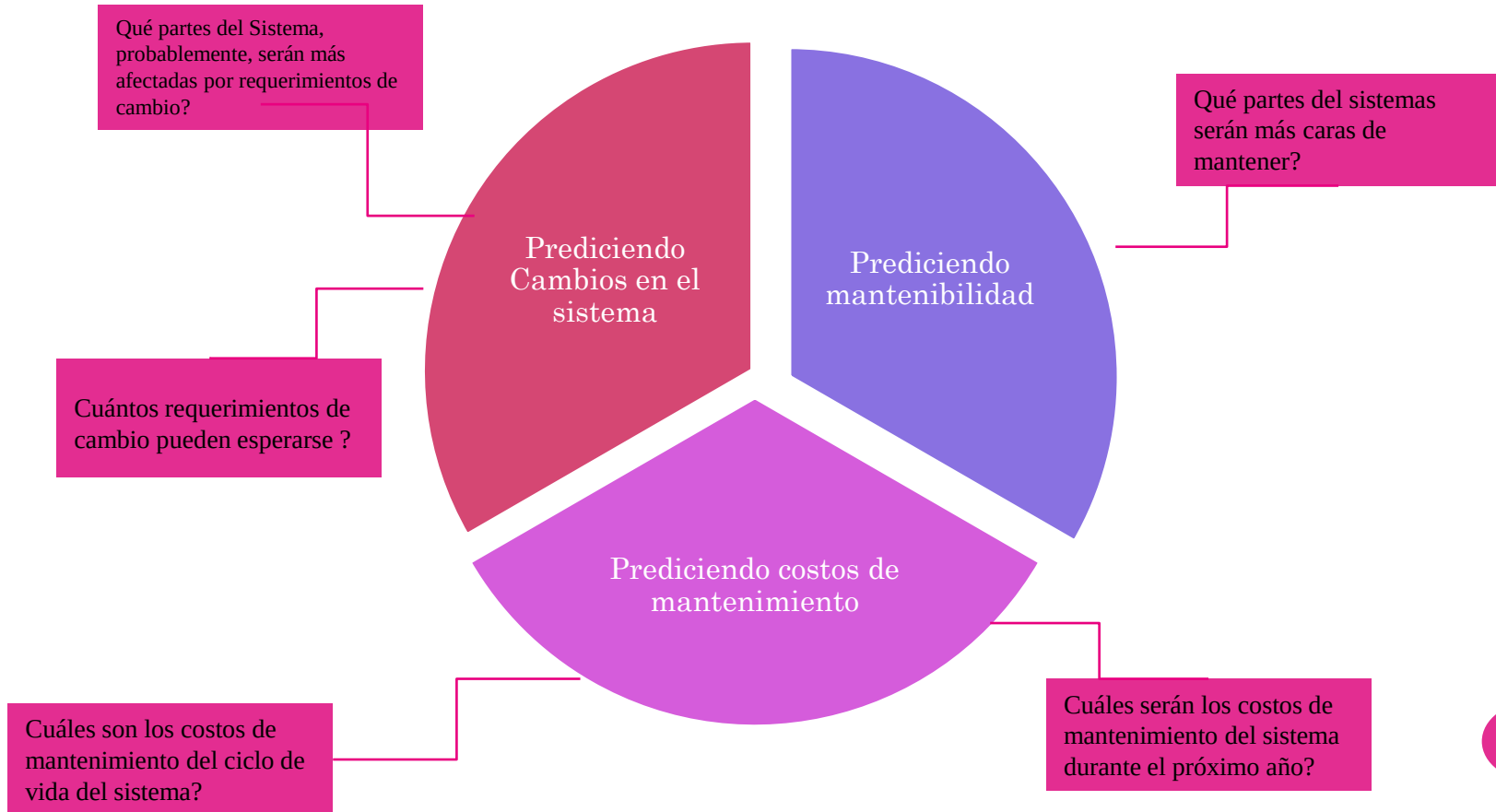
Predicción del Mantenimiento

✦ La predicción del mantenimiento está relacionada con la evaluación de cuáles son las partes del sistema que pueden causar problemas y tener costos altos de mantenimiento:

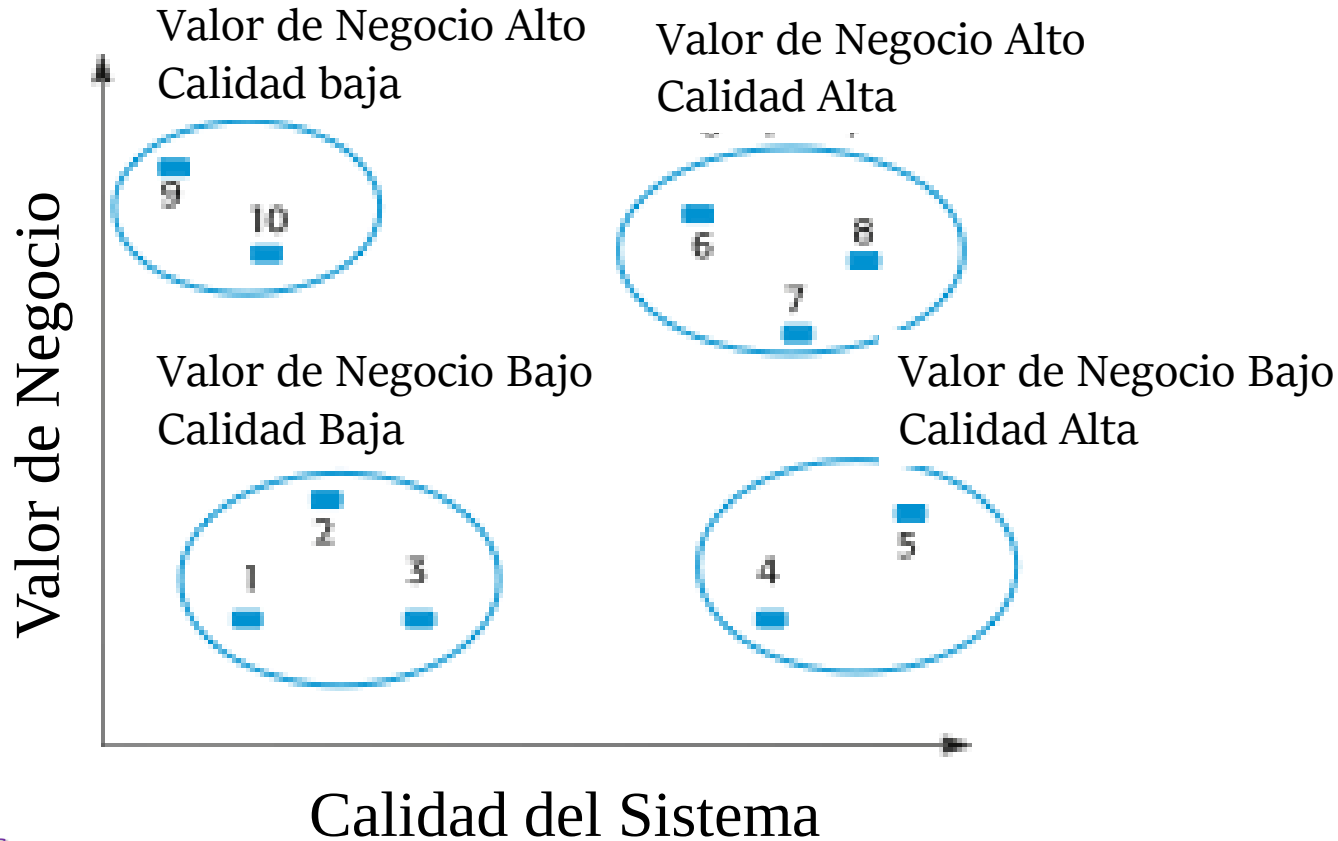
- La aceptación del cambio depende de la mantenibilidad de los componentes afectados por el cambio.
- La implementación de los cambios degrada el sistema y reduce su mantenibilidad
- Los costos de mantenimiento dependen del número de cambios y los costos del cambio dependen de la mantenibilidad.



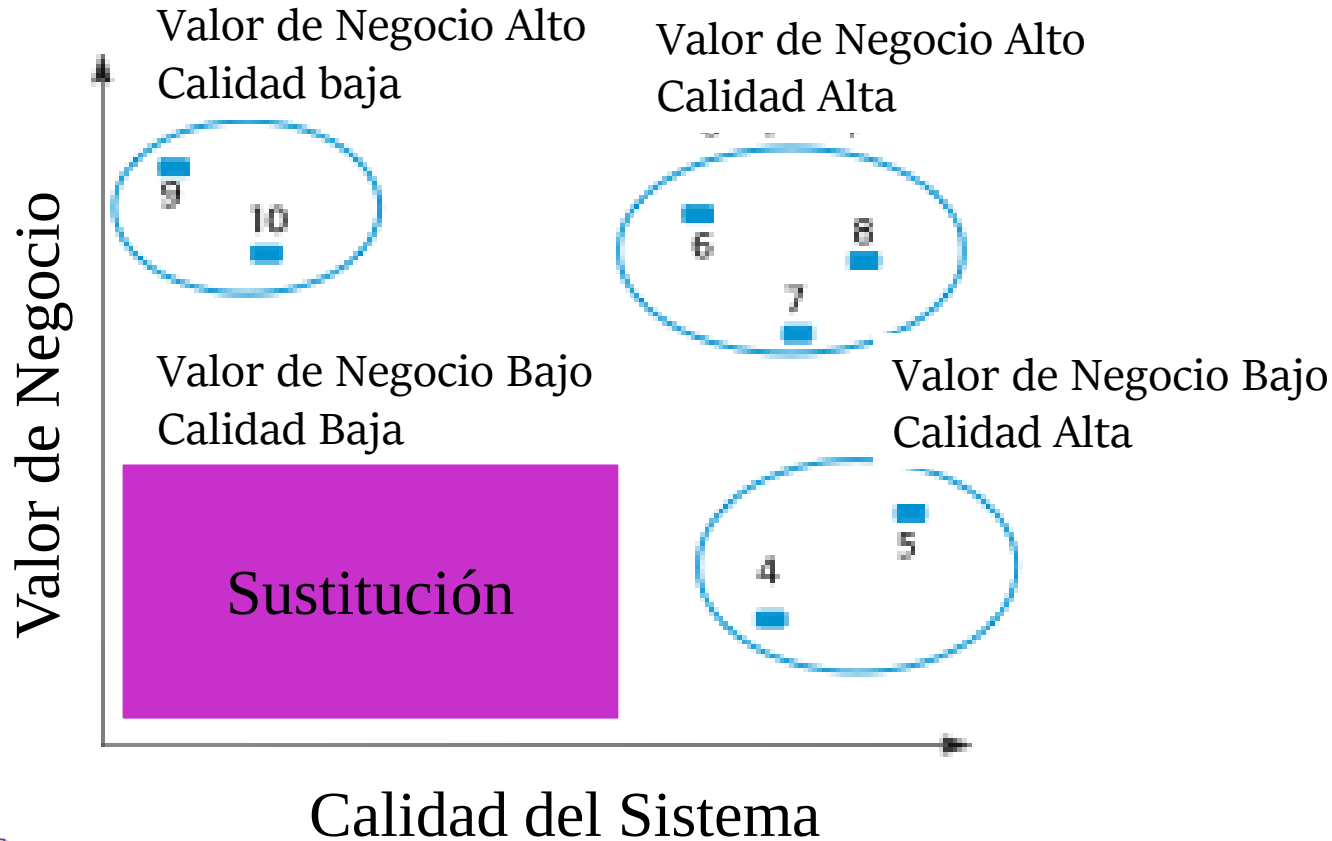
Predicción del Mantenimiento



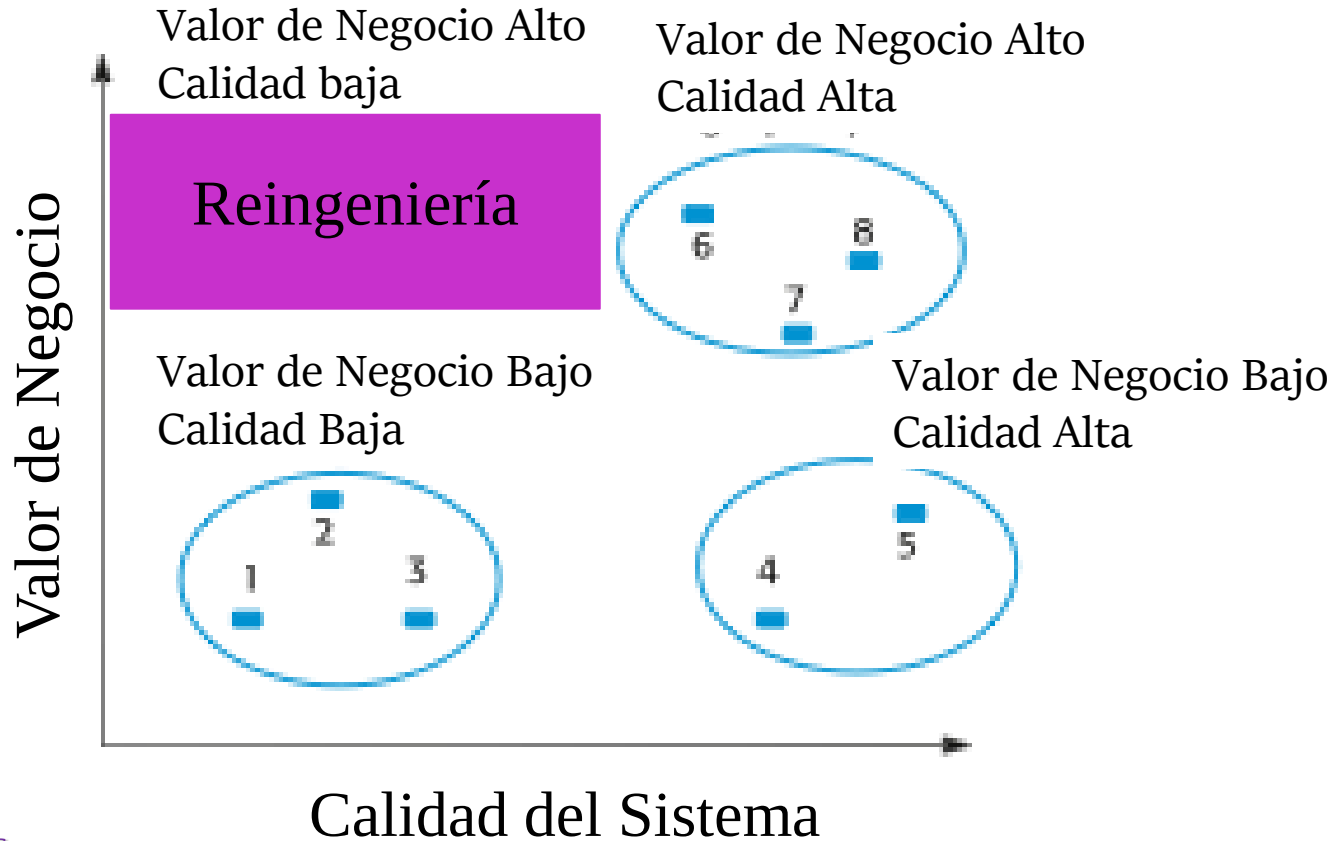
Ejemplo de Valoración de un sistema heredado



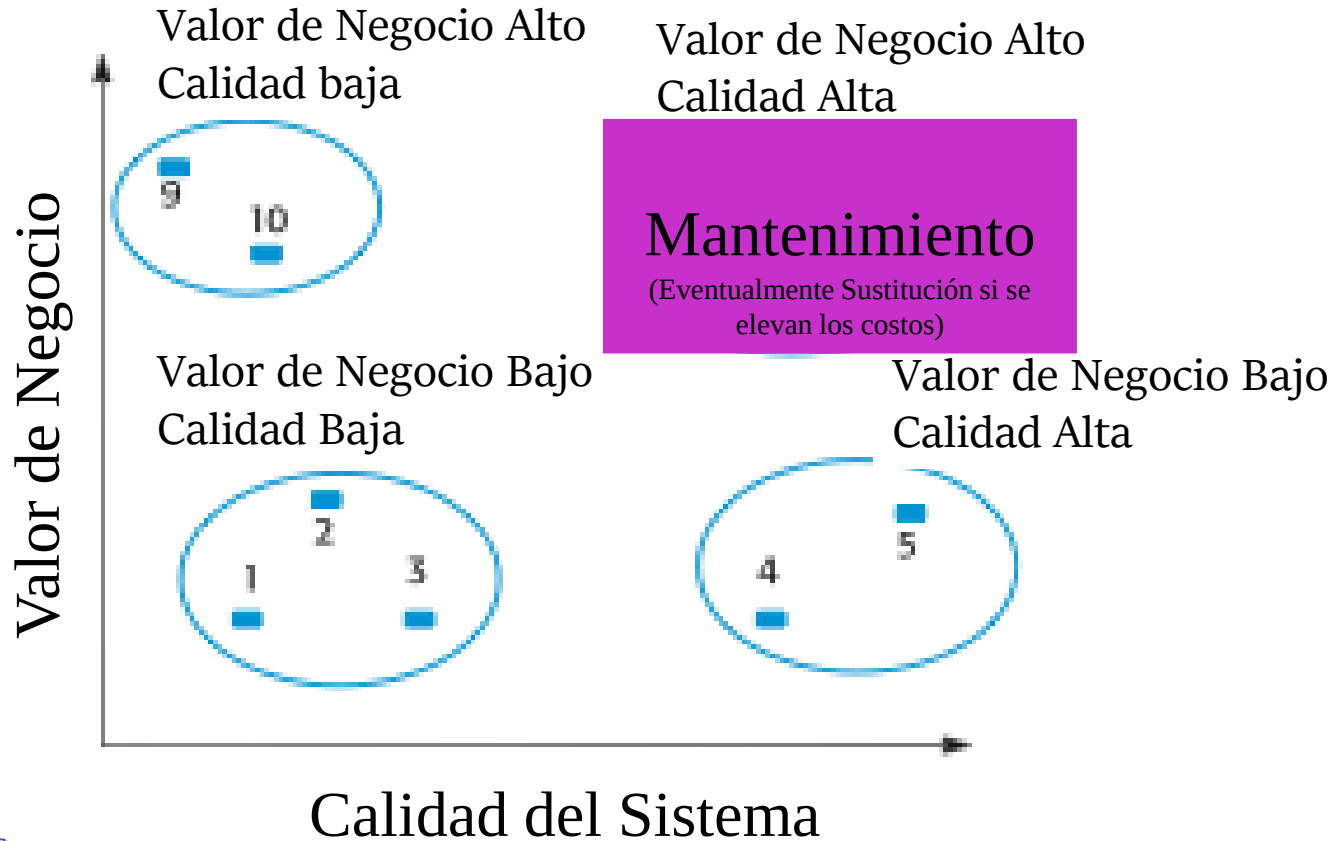
Ejemplo de Valoración de un sistema heredado



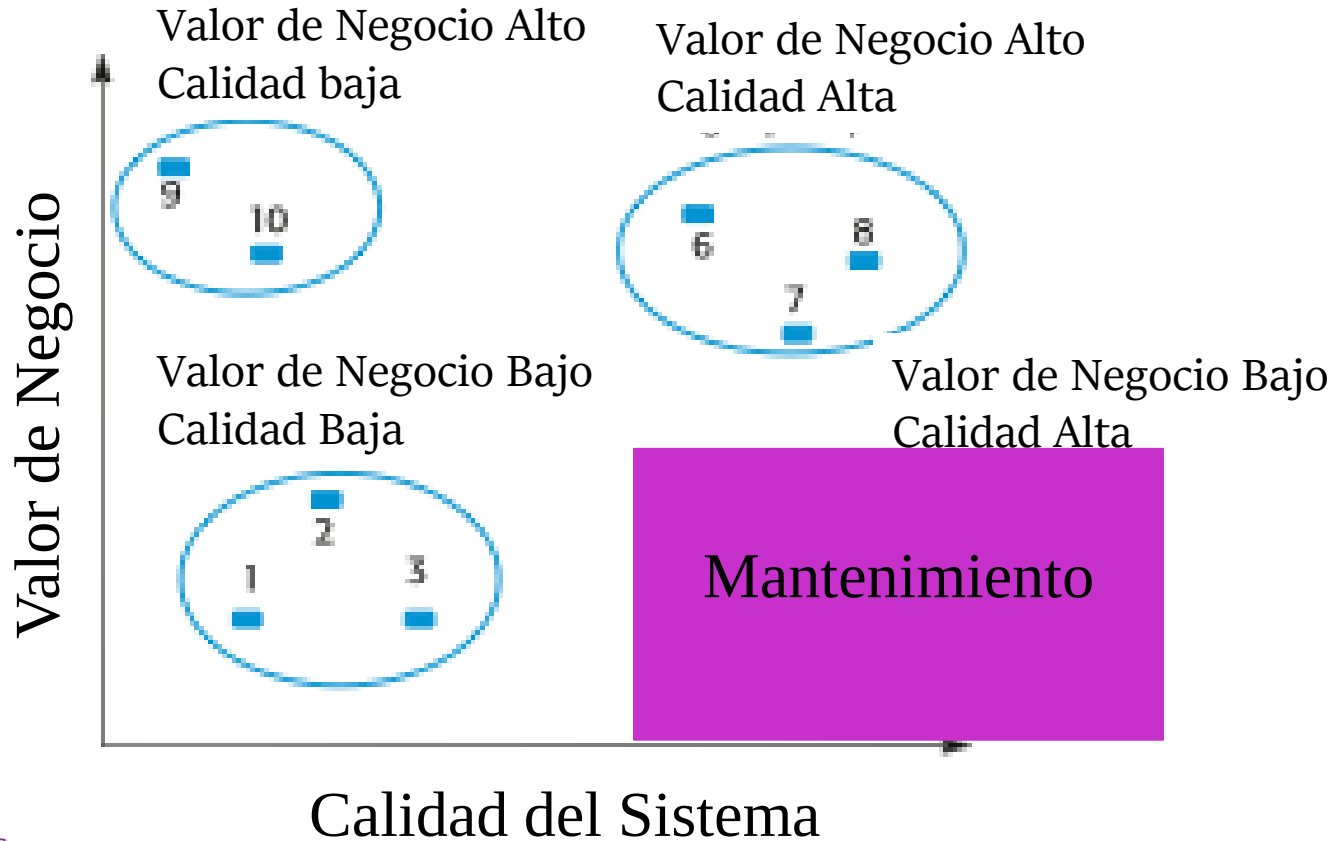
Ejemplo de Valoración de un sistema heredado



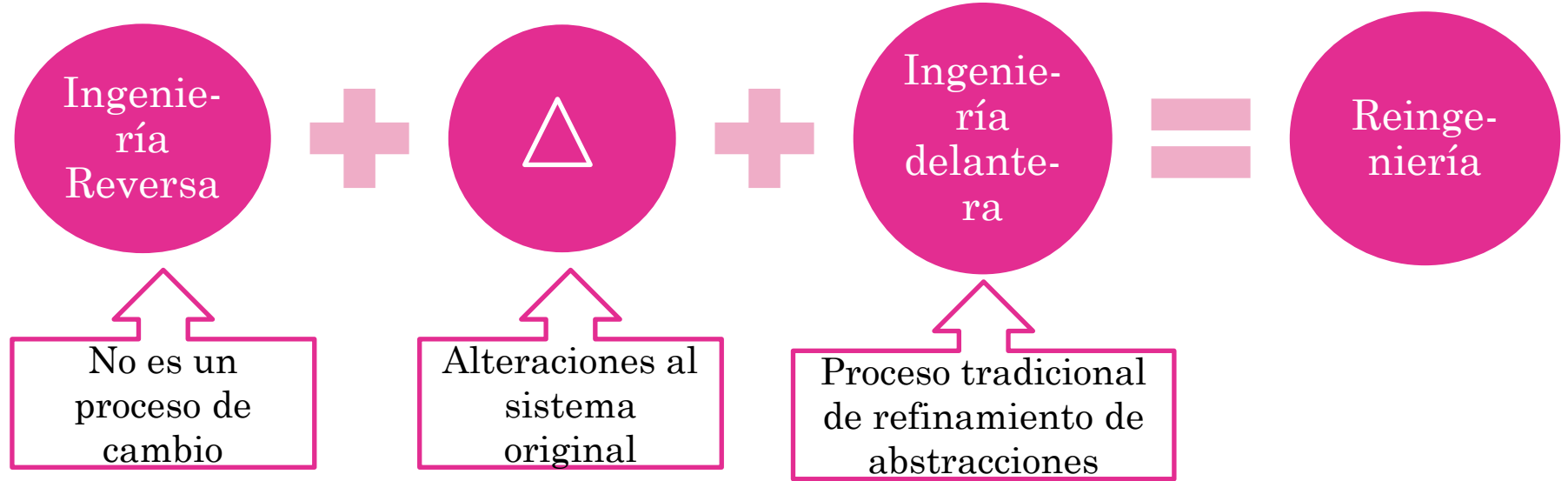
Ejemplo de Valoración de un sistema heredado



Ejemplo de Valoración de un sistema heredado



Reingeniería de Software

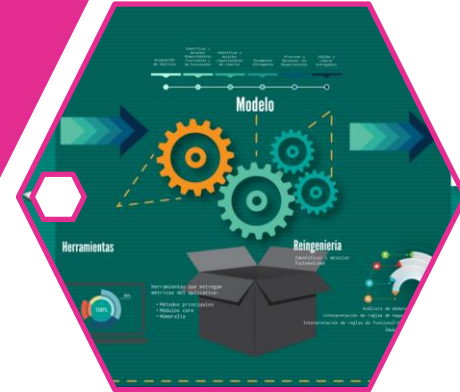


Estrategias de Evolución del Software



Mantenimiento

Reingeniería



Reingeniería de Software: Tópicos

Traducción de Código Fuente

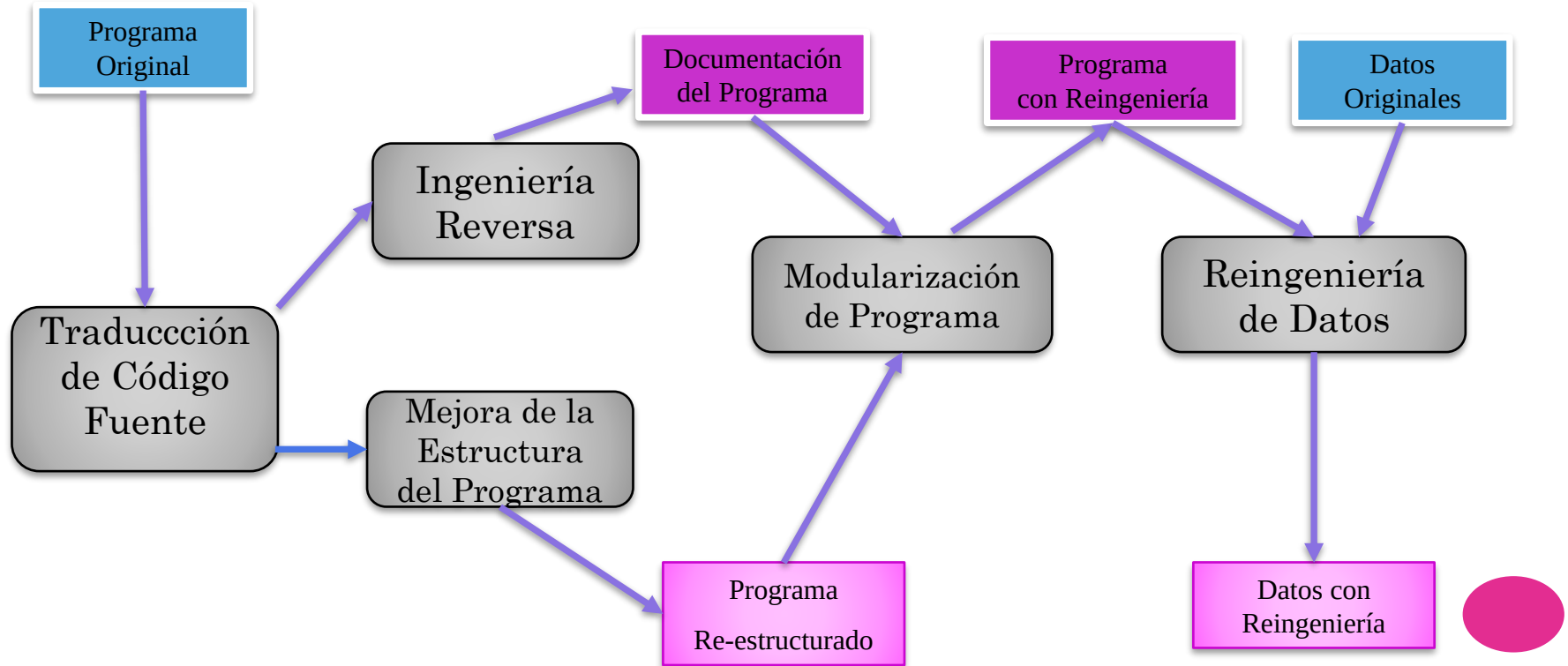
Ingeniería Reversa

Mejora de Estructura de Programa

Modularización del Programa

Re-Ingeniería de Datos

El Proceso de Re-ingeniería



Ventajas de la Re-ingeniería



Reducir Riesgos

Existe un alto riesgo en el desarrollo de nuevo software. Puede haber problemas de desarrollo, de personal, de especificación.



Reducir Costos

El costo de la re-ingeniería es significativamente menor que el costo del desarrollo de nuevo software.



Factores de Costo de la re-ingeniería



La calidad de software al que se le va a aplicar re-ingeniería.



La herramienta de soporte disponible.



La extensión de la conversión de datos que se requiere.



La disponibilidad de personal experto para realizar la re-ingeniería.



Factores de Complejidad



Complejidad de Control : puede medirse examinándose las sentencias condicionales en el programa.



Complejidad de Datos: La complejidad de las estructuras de datos y los componentes de interfaces.



Tamaño de los módulos



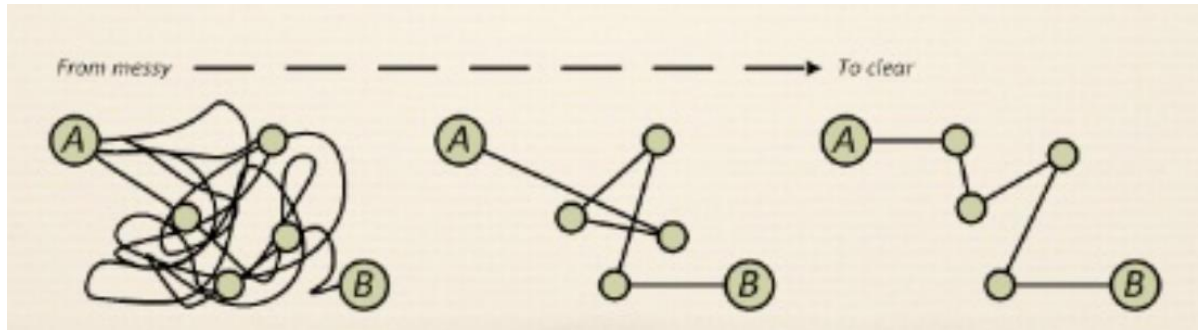
Extensión de los nombres identificadores: Los nombres más largos implican más legibilidad.



Comentarios del Programa: Tal vez más comentarios significan mantenimiento más fácil.

¿Y el Refactoring....?

- ❖ Es una forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos.
- ❖ Es una transformación del software que:
 - Preserva la estructura externa
 - Mejora la estructura interna

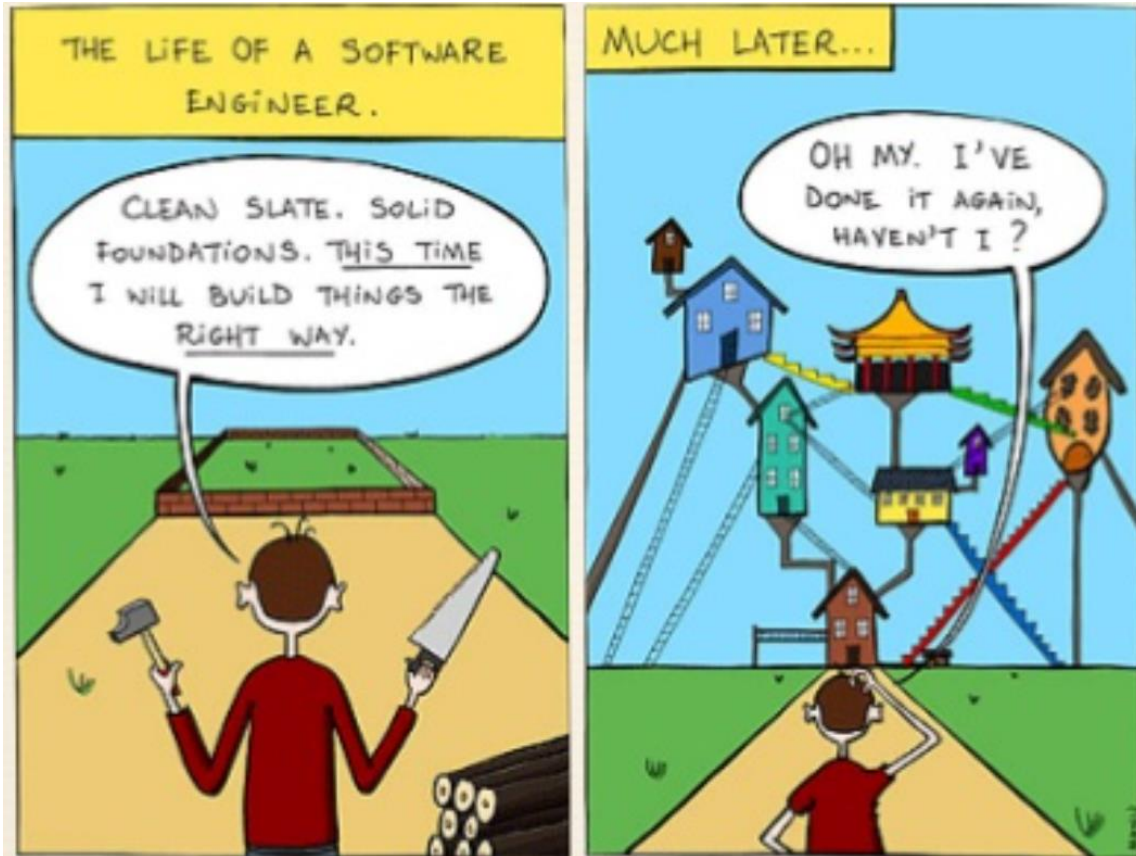


El Refactoring...

- ❖ Son cambios que se realizan en el sistema que:
 - No cambien el comportamiento observable (todas las pruebas aún pasan)
 - Eliminan la duplicación o la complejidad innecesaria
 - Mejoran la calidad del software
 - Hacen que el código sea más simple y más fácil de entender
 - Flexibiliza el código
 - Hace que el código sea más fácil de cambiar



¿Por qué es necesario hacer Refactoring?

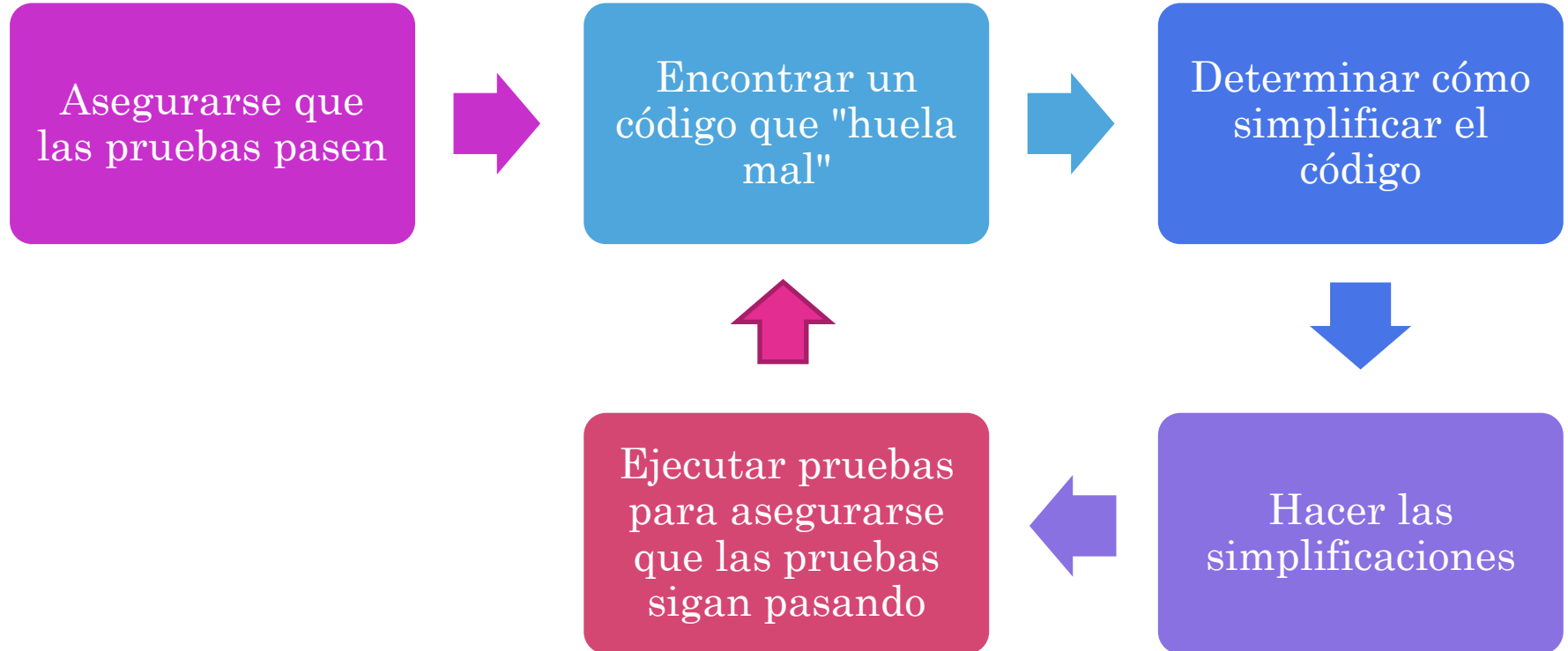


¿Por qué es necesario hacer Refactoring?

- ❖ Evitar el "deterioro del diseño"
- ❖ Limpiar desorden en el código
- ❖ Simplificar el código
- ❖ Incrementar la legibilidad y la comprensibilidad
- ❖ Encontrar errores
- ❖ Reducir el tiempo de depuración
- ❖ Incorporar el aprendizaje que hacemos sobre la aplicación
- ❖ Rehacer las cosas es fundamental en todo proceso creativo



¿Cómo hacer Refactoring?



Refactoring en contexto de Desarrollo y Evolución del Software

- ❖ Refactoring y TDD
- ❖ Refactoring como mantenimiento preventivo



Bibliografía de referencia

- ❖ **Sommerville, Ian** - INGENIERÍA DE SOFTWARE - Novena Edición (Editorial Addison-Wesley Año 2011).
- ❖ **Priyadarshi, Tripathy & Kshirasagar, Naik** - SOFTWARE EVOLUTION AND MAINTENANCE - A PRACTITIONER'S APPROACH
- ❖ **Beck, Kent** - EXTREME PROGRAMMING EXPLAINED

